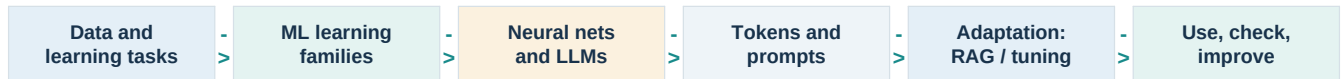


MACHINE LEARNING AND GENERATIVE AI

A compact, step-by-step study guide from core ML ideas to practical prompt design

How to use this guide Read one part at a time. Follow the diagrams, remember the bold takeaway, then test yourself with the review questions. Examples use familiar tasks such as spam detection, house prices, chess, and course creation.

The learning path



Start with how a model learns; finish with how to use and evaluate one responsibly.

At a glance

Part	You will learn
1. Foundations	AI vs ML vs deep learning; examples, features, labels, fitting, and data splits.
2. Learning types	Supervised, unsupervised, self-supervised, and reinforcement learning.
3. Neural networks	Layers, weights, training, and what a model's learned parameters represent.
4. Generative AI	Foundation models, LLMs, tokens, transformers, and next-token generation.
5. Improve answers	Prompting, retrieval (RAG), fine-tuning, and generated knowledge.
6. Use it well	Limitations, brainstorm use cases, prompt methods, and reusable workflows.

The one-sentence idea Machine learning learns patterns from examples; generative AI uses learned patterns to create new content, guided by prompts and sometimes by retrieved information.

PART 1

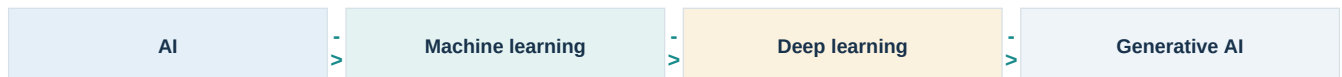
AI and machine-learning foundations

The basic vocabulary used throughout the rest of the guide.

1. How the terms fit together

Artificial intelligence (AI) is the broad field of building systems that perform tasks associated with intelligent behavior.

Machine learning (ML) is one approach within AI: a system learns patterns from data instead of relying only on hand-written rules. **Deep learning (DL)** is ML that uses neural networks with many layers. **Generative AI** creates new content, such as text, images, audio, or video; many modern generative systems use deep learning.



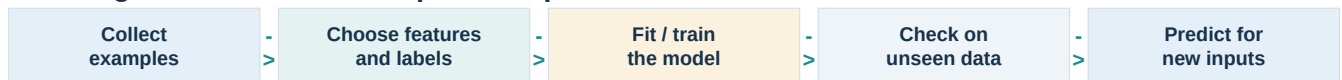
A useful simplified hierarchy. Generative AI describes what a system does; deep learning describes a common way it is built.

2. The data vocabulary

Term	Plain meaning	Example: spam email
Example / row	One recorded case	One email
Feature / input	A clue supplied to the model; often written X	Sender, subject words, links
Label / target	The answer to learn; often written y	Spam or not spam
Dataset	A collection of examples	Many emails and their labels
Model	A learned pattern or rule that maps inputs to outputs	A spam classifier
Prediction	The model's output for a new input	Spam: yes, with a score

A feature is information used to make a prediction. A label is the known answer used during supervised training. At prediction time, the new example usually has features but no known label yet.

3. Fitting a model: from examples to a prediction



Training is the learning phase. Evaluation checks whether the learned pattern works beyond the examples used to fit it.

An **unfitted model** is an algorithm/model setup whose useful parameters have not yet been learned from the training data. A **fitted model** has gone through training and can make predictions. Fitting does not mean the model is correct; it means training has run.

Data split	Purpose	Simple rule
Training set	Learn model parameters	Use it to fit the model.
Validation set	Compare choices and tune settings	Do not use it as the final unbiased score.
Test set	Estimate performance on unseen data	Keep it separate until final evaluation.

Remember A model learns a mapping: features (X) -> predicted target (y). The goal is not to memorize training rows, but to generalize to useful new cases.

PART 2

Four ways machine-learning systems learn

Choose the learning family by asking what feedback the learner receives.

Family	What feedback is available?	Everyday example
Supervised	Inputs paired with correct target labels	Learn from emails marked spam / not spam.
Unsupervised	Inputs without target labels	Group customers by similar purchase patterns.
Self-supervised	Training targets are created from the data itself	Hide a word or next token and predict it.
Reinforcement	Actions receive rewards or penalties over time	Learn chess moves from wins, losses, and search.

4. Supervised learning: learn from labeled examples

The model sees input-output pairs and learns a mapping from the input to the target. Two common supervised task types are **classification** and **regression**.

Task	Output	Example	Common methods
Binary classification	One of two classes	Spam / not spam; dog / not dog	Logistic regression, SVM, trees, neural nets
Multiclass classification	One of 3+ classes	Cat, dog, or bird	SVM, trees, neural nets
Regression	A numeric value	Predict a house price	Linear or polynomial regression, trees

Important correction: logistic regression is commonly used for classification, despite the word regression in its name. Polynomial regression models curved relationships by adding polynomial features; it remains a regression task when the target is a continuous number. SVM means Support Vector Machine; SVMs can do both classification and regression.

Shallow vs deep is a different distinction. It describes model architecture or depth, not a classification subtype. An SVM is often considered a traditional, non-deep-learning method; neural networks with many learned layers are deep learning.

5. Unsupervised learning: find structure without target labels

Method	What it does	Example
Clustering	Groups examples that are similar under a chosen measure	Group shoppers by behavior; discover topic groups.
Dimensionality reduction	Compresses many features into fewer useful dimensions	Plot high-dimensional data in 2D to inspect patterns.

Unsupervised does not mean untrained. The system still fits patterns to data; it simply does not receive the target labels used in supervised learning. Clusters are not automatically meaningful - a person must interpret and validate them.

6. Self-supervised learning: create the training target from the data

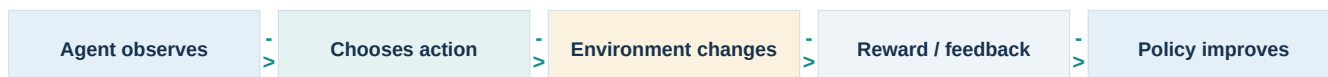
In self-supervised learning, the data provides a training signal without humans labeling every example. A language model might hide a word and predict it, or predict the next token in a sequence. The model is still optimized against a target; the target was generated from the original text.



Masked language modeling predicts a hidden token using surrounding context. Causal language modeling predicts the next token from earlier context.

7. Reinforcement learning: learn by acting and receiving rewards

A learner called an **agent** interacts with an **environment**. It observes a situation, chooses an action, and receives a reward. Over many trials it can learn a **policy** - a strategy for choosing actions. The aim is often to maximize total reward over time, including future rewards.



Chess example: board position -> move -> new position -> win/loss or intermediate reward. A winning result may arrive only after many moves.

Trial and error means trying actions, observing their consequences, and improving future choices. Search or simulated self-play can help in games such as chess, but trial-and-error reward learning is the core RL idea.

Quick recall Labels? Supervised. No target labels? Unsupervised. Targets made from the data? Self-supervised. Rewards after actions? Reinforcement learning.

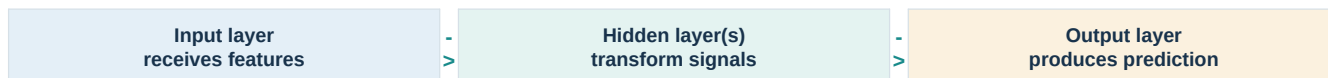
PART 3

Neural networks and deep learning

A neural network is a mathematical model made from connected computations, not a tiny human brain.

8. The three familiar layer groups

A basic feed-forward neural network is often introduced with three layer groups:



The number of hidden layers varies. Deep learning typically uses neural networks with multiple learned layers.

Part	What it does	House-price example
Input	Receives numeric or encoded features	Size, neighborhood, age
Hidden	Combines signals using weights, biases, and activation functions	Learns patterns such as size interacting with location
Output	Produces a prediction in the task's required form	Estimated price

9. How training changes the network

- 1. Forward pass.** Features move through the layers and produce a prediction.
- 2. Measure error.** A loss function compares the prediction with the known target.
- 3. Compute gradients.** Backpropagation calculates how parameters contributed to the error.
- 4. Update parameters.** An optimizer uses those gradients to adjust weights and biases.
- 5. Repeat and evaluate.** Training repeats over examples; validation data helps check generalization.

The **weights and biases** are numerical parameters. Training changes them so the network's predictions improve on its objective. Backpropagation computes gradients; the optimizer uses them to update parameters. These are related but different steps.

Tiny neuron example Suppose a unit receives an input value of 2, multiplies it by a learned weight of 3, and adds a bias of 1. Its value before an activation function is $2 \times 3 + 1 = 7$. Real networks repeat and combine many such operations.

Correcting a common phrase Neural networks are a main building block of many generative AI models. A model's learned patterns are encoded across its parameters; it is not simply a database that stores each training sentence as a retrievable fact.

10. Shallow learning and deep learning

A useful beginner contrast is traditional ML methods such as linear models, decision trees, or SVMs versus deep neural networks with many layers. Both are machine learning. Which one is best depends on the data, task, amount of data, compute, and need for interpretability; 'deep' does not automatically mean better.

PART 4

Generative AI, foundation models, and LLMs

Connect the neural-network idea to the systems that create text and other media.

11. What makes AI generative?

A **generative model** learns patterns in training data and uses them to produce new outputs that fit a request or context. Output can be text, code, images, audio, or video. Generation is not the same as guaranteed retrieval of a true fact.

Keep the categories separate Supervised, unsupervised, self-supervised, and reinforcement learning describe learning signals. Generative AI describes a system's ability to produce content. A generative model can use self-supervised pretraining and additional adaptation steps.

12. Foundation models and large language models

Term	Beginner meaning	Example
Foundation model	A broadly pretrained model that can be adapted or prompted for many tasks	One model used for writing, summarizing, and question answering
LLM	A model trained to process and generate language at large scale	A chat assistant drafting an explanation
Domain-specific model	A model adapted or designed for a narrower field or task	A system focused on legal-document classification

General-purpose models can handle a wider range of requests. Domain-specific systems may be better suited to a focused vocabulary or workflow, especially when supported by reliable domain data. Neither type is automatically accurate.

13. Open source, open weights, and closed models

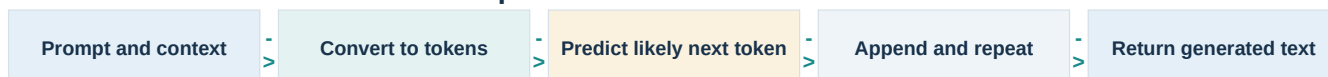
Label	What may be available	What to check
Open-source AI	The term can cover code, model parameters, data information, and usage rights	Check the actual license and what components are released.
Open-weight model	Model parameters can be downloaded or run by users	Open weights do not automatically mean all training data, code, or unrestricted rights are open.
Closed / hosted model	Access commonly comes through a provider's API or application	Check service terms, available controls, cost, and data handling.

Remember 'Available to download' describes access to weights. It does not by itself prove the complete AI system meets an open-source definition.

14. Transformers and attention

Many modern LLMs use the **Transformer** architecture. Attention helps a model weigh relationships among tokens in its context, such as which earlier words matter for interpreting a current word. It is a learned computation, not human-style attention or understanding.

15. How an LLM writes: next-token prediction



At each generation step, the model estimates possible next tokens. Decoding settings affect how a token is selected.

An LLM generates text one token at a time. It uses the prompt and earlier generated tokens to estimate what should come next. A token may be a whole word, part of a word, punctuation, or another unit. A fluent continuation can still contain a false claim.

16. High-level training stages

Stage	Purpose	Simple description
Pretraining	Learn broad patterns from large datasets	For a causal language model, predict missing next-token targets.
Instruction tuning	Improve following task instructions	Train on examples of requests and useful responses.
Preference / alignment work	Shape responses toward chosen quality and safety goals	Use feedback or comparisons, with limitations.
Use-time prompting / tools	Guide a trained model for this request	Supply instructions, context, examples, or retrieve documents.

PART 5

Tokens, context windows, and prompt building

Prompts guide a model at use time; they do not automatically retrain it.

17. Prompt, token, tokenizer, context window

Word	Meaning	Easy picture
Prompt	The instructions and information supplied to guide an answer	A task card for the model
Token	A piece of text processed by a model; it may be a word, word part, or symbol	Small text building block
Tokenizer	The software that splits text into tokens and maps them to IDs	A text-to-number converter
Context window	The amount of token context a model can handle for a request	A limited workbench for prompt, history, and other context

Token totals can include the system and user instructions, conversation history, supplied documents, retrieved passages, and generated output. Exact counting rules and limits vary by model. Long conversations and documents can use up room that would otherwise be available for new information or the answer.

18. A prompt's main building blocks

Block	Ask	Example
Instruction	What should it do?	Explain overfitting.
Context	What background or purpose matters?	I am a beginner learning classification.
Example	Can I demonstrate the desired pattern?	Show one sample input and answer.
Persona	What perspective may help?	Answer as a patient tutor.
Format	What shape should the result have?	Three bullets with one example.
Tone	How should it sound?	Friendly and informative.

Memory aid What? (instruction) Why / for whom? (context) Show? (example) Who's perspective? (persona) Shape? (format) Voice? (tone). Use only the blocks that help.

19. Build one prompt step by step

- 1. State the task.** Use a direct action: explain, compare, summarize, draft, classify, or plan.
- 2. Add useful context.** Name the audience, purpose, background, or source material that affects the answer.
- 3. Set scope and boundaries.** Say what to include, omit, assume, or do when information is missing.
- 4. Give just enough information.** For code, include language, framework, relevant snippets, required behavior, and constraints. Put essential information first.
- 5. Choose structure and tone.** Request a table, ordered steps, length, and voice if those matter.
- 6. Review and refine.** Check facts and whether the response followed the request; give specific correction instructions.

Reusable example

Prompt Explain [concept] to [audience] for [purpose]. Include [must-have details] and leave out [out-of-scope details]. Use [format], keep it around [length], and use a [tone] tone. If a fact is uncertain or missing, say so rather than inventing it.

Examples can clarify a hard-to-describe format or style. Few-shot prompting means placing a few input-output demonstrations in the prompt, then giving a new query. Examples show a pattern; they do not supply missing facts or guarantee the model will follow the pattern.

PART 6

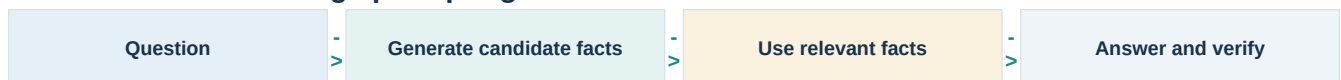
Improve or supply knowledge

Three different levers: prompt the model, retrieve information, or change model weights.

20. Contextual prompting

Contextual prompting means putting useful background in the prompt: definitions, constraints, examples, or relevant text. The model can use this context for the current answer. Prompting itself does not update model weights or permanently teach the model.

21. Generated-knowledge prompting



A two-step idea: first ask the model to produce question-related knowledge, then use it to answer.

This can help organize possible facts or commonsense ideas. But the generated facts come from the model and can be incorrect. It is not the same as searching an external, verified knowledge source. A research method with this name generates knowledge first and then supplies it as added input for answering (Liu et al., 2022).

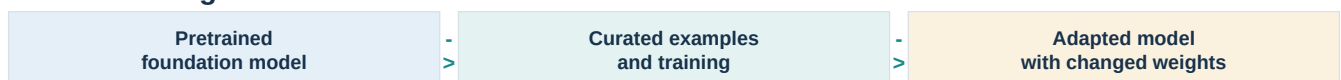
22. Retrieval-Augmented Generation (RAG)



RAG retrieves information at answer time; retrieval quality and answer quality both need checking.

RAG can help with information that is current, specialized, or stored in selected documents. Information can be pasted by a person or fetched automatically from an indexed source. The model's weights are not updated by retrieval. RAG can still fail if the source is stale, permissions are wrong, search misses a relevant passage, or the model misuses the passage. Search also adds latency and retrieved text uses context tokens (Lewis et al., 2020).

23. Fine-tuning



Fine-tuning continues training a pretrained model. Depending on the model and service, it may be available through an API or local tools.

Fine-tuning can adapt response style, task behavior, or a repeated format. It requires suitable example data and evaluation. A tuned model may need less repeated instruction in its prompt, but reduced latency or cost is not guaranteed. Fine-tuning is not usually the best way to keep frequently changing facts current; a retrieval source is often more appropriate.

24. Choose the right approach

Approach	Best for	Does it change weights?	Main caution
Prompting	One request, constraints, examples, or brief supplied context	No	Clear instructions do not guarantee truth.
RAG	Current or source-specific facts and documents	No	Retrieval can miss, retrieve poorly, or expose data if access is misconfigured.
Fine-tuning	Repeated behavior, style, or task patterns	Yes	Needs quality examples and evaluation; facts may still go stale.

Fast decision Need a clearer instruction? Prompt. Need facts from documents? RAG. Need a repeated behavior or format learned from examples? Consider fine-tuning. These approaches can also be combined.

25. Knowledge cutoff

A model's training data has an approximate knowledge cutoff. It may not know later events or updates. A connected search or RAG system can supply newer information to a response, but this does not mean the base model learned it permanently. Always check current details against a current source.

PART 7

Limits, risks, and multimodal generation

A capable model can still be wrong, inconsistent, or unsuitable for a particular decision.

Challenge	Meaning and example	Practical response
Hallucination	A confident-sounding but false or unsupported answer; for example, a fabricated citation.	Verify important facts and citations against reliable sources.
Bias and fairness	Patterns in data or design can lead to unequal results across groups; for example, a screening system favoring proxy signals.	Test across relevant groups, review outcomes, and keep oversight for consequential decisions.
Privacy, security, misuse	Sensitive information may be handled unexpectedly; prompt injection can try to steer a system; outputs can be misused.	Use approved services, limit access, treat retrieved content as untrusted, and review actions.
Structured data	Models can misread tables, make arithmetic errors, or return malformed JSON.	Use calculators/code for exact work; validate structured output against a schema.
Nondeterminism	The same prompt may produce different outputs, especially when generation samples among likely tokens.	Save results and settings when consistency matters; verify output.
Prompt sensitivity	Small wording or ordering changes can affect the answer.	State task, audience, constraints, and format clearly; test representative prompts.
Common-sense gaps	The model may miss real-world constraints or unstated assumptions.	State important constraints and check advice that affects health, safety, or money.
Knowledge cutoff	Training may not include recent or private information.	Supply the facts or retrieve them from a current, trusted source.

A useful rule A fluent answer is not proof. For important work, provide reliable context, check the output, and use a suitable tool for exact calculations or actions.

26. Text, image, video, and audio

Direction	What it means	Example
Text-to-text	Text prompt creates text	Explain a topic or draft an email
Text-to-image	Text description creates an image	Create an illustration for a lesson
Text-to-video	Text description creates video	Make a short scene for an introduction
Text-to-audio	Text creates spoken audio, music, or effects	Generate a spoken narration
Multimodal input	System takes image, audio, or video and responds	Describe a picture or summarize a recording

A single campaign might use AI to draft text, create a poster, generate a short video, and make a voice-over. Review accuracy, rights, audience fit, and quality for every output type.

PART 8

Using AI well: brainstorm, prompt, evaluate, reuse

Move from a one-off request to a repeatable human-reviewed workflow.

27. GenAI as a brainstorming assistant

Use	Helpful role	Keep in mind
Brainstorming	Generate ideas, options, titles, and alternatives	Treat ideas as candidates; choose and check them.
Summarization	Condense documents or conversations	Check names, numbers, and important nuance against the source.
Text enhancement	Improve clarity, grammar, or tone for a near-final draft	You approve the meaning and final version.
Break down work / code	Turn a large task into steps and draft code	Test code and inspect security and edge cases.
Content framework	Build an outline, lesson structure, or first draft	Use the framework as a starting point, not verified content.
Images on demand	Make or revise an image from a description	Review the result for accuracy and suitability.
AI features in apps	Natural-language search, drafting, or document-based support	Limit data/tool access and evaluate the feature.

28. Prompt best practices

1. **Be specific.** Name the deliverable and success criteria.
2. **Give context.** Include the audience, purpose, and relevant background.
3. **Set scope.** Say what to include, exclude, or do when information is missing.
4. **Specify format.** Ask for the structure: table, steps, bullets, or word limit.
5. **Protect private information.** Do not include confidential data in an unapproved service; anonymize where possible.
6. **Ask for clarification.** Request simpler wording, definitions, or stated assumptions.
7. **Compare perspectives.** Ask for alternatives and trade-offs; do not assume each is equally supported.
8. **Split complex work.** Use smaller tasks with clear outputs and checks.
9. **Iterate.** Say specifically what to change and what to preserve.

29. Prompt-engineering methods

Method	Use it when	Example
Task decomposition	A request is too large or contains distinct jobs	Outline course -> draft one chapter -> create quiz.
Ask for key steps	You need a checkable explanation or intermediate calculation	Give the result, key steps, assumptions, and a check.
Prompt chaining	The output from one step should shape the next	Brainstorm -> select -> outline -> draft -> review.
Interactive prompting	You want to steer through questions and revisions	Review a draft, answer clarifying questions, request changes.
Generated knowledge	Candidate background ideas may help before answering	Generate likely relevant facts, then answer and verify them.
RAG / external knowledge	The answer needs current, private, or domain-specific sources	Retrieve approved policy pages, then answer with source references.

Requesting steps can help structure an answer, but an explanation is not a guarantee of correctness or a perfect record of the model's internal process. Chain-of-thought research reports gains on some reasoning tasks (Wei et al., 2022); faithfulness of written explanations remains a separate question (Lanham et al., 2023).

30. Build a reusable prompt system

A reusable prompt template stores stable instructions and leaves changing fields as placeholders. It is useful when the same kind of task happens often, such as creating lessons for a course.



Human-AI collaboration: a person defines the goal, AI drafts, a person checks, AI revises, and a person approves.

Six-stage roadmap

Stage	Action	Course-creation example
1. Goal	Describe one useful task in one concise objective	Create a beginner lesson on supervised learning.
2. Requirements	Clarify audience, context, must-haves, sources, and constraints	For adults new to AI; include a spam example.
3. Scope	Keep the prompt manageable and substantial enough to reuse	Create one lesson, not an entire course in one prompt.
4. Output design	Choose structure, tone, length, and presentation rules	Objectives, explanation, example, practice, recap.
5. Test	Run the prompt on a representative topic and inspect the output	Check for missing facts, confusing terms, and length.
6. Improve / reuse	Repair recurring issues, save the version, and use it again	Add a source-check instruction if facts were invented.

Reusable course-lesson prompt

Template Create a lesson about [TOPIC] for [AUDIENCE]. Goal: help learners [LEARNING OUTCOME]. Assume they already know [PRIOR KNOWLEDGE]. Use [APPROVED SOURCE] for factual claims and flag missing or uncertain information. Include: [REQUIRED SECTIONS]. Structure the answer as [FORMAT], use a [TONE] tone, and keep it near [LENGTH]. Finish with [PRACTICE OR REVIEW].

Diagnose the draft, then repair the template

Output problem	Likely template repair
Missing information or changed facts	Name must-have content, supply approved sources, and ask it to flag uncertainty.
Instructions misunderstood	Rewrite the objective as one direct action; add an example if needed.
Structure unclear or length wrong	Specify headings, order, format, and approximate length.
Tone misses the audience	Describe who the audience is and the voice they need.
Formatting inconsistent / visuals hard to read	Set shared style rules and ask for a readability check.
Same manual correction every time	Add the fix to the template, test it, and save a new version.

Practical payoff A tested prompt template can reduce repeated setup work and make outputs more consistent. Quality improves when requirements are clear and a person checks the result; reusing a template alone does not guarantee quality.

Worked example: turn a vague request into a reusable prompt

Weak request	Improved template
Make a course about AI.	Create a 4-lesson introduction to AI for adult office staff with no technical background. Each lesson should include one learning goal, a plain-language explanation, one workplace example, and three review questions. Use the supplied course notes for factual claims. Flag gaps instead of inventing details.

Test it: run the template on one topic. Check that the facts match the notes, the examples fit the audience, every requested section is present, and the lesson is the right length. Turn any repeated correction into a prompt rule, then test the revised version on a second topic.

REVIEW

One-page recall sheet

Use this page to quickly retrieve the main ideas before a quiz or study session.

Core map Labeled examples -> supervised. No target labels -> unsupervised. Targets derived from data -> self-supervised. Actions plus rewards -> reinforcement learning. Prompt guides; RAG retrieves; fine-tuning changes weights.

Self-check: try answering before reading the key

1. In a house-price dataset, name two features and the target.
2. Why is logistic regression usually a classification algorithm?
3. What is the difference between a cluster and a class label?
4. What role does reward play in a chess-learning agent?
5. What gets counted toward a model's context budget?
6. Why can a model sound certain and still hallucinate?
7. When would RAG be a better fit than fine-tuning?
8. What are the six useful prompt building blocks?
9. How is prompt chaining different from a single prompt?
10. What should a human do before approving a reusable AI-generated lesson?

Answer key

#	Answer
1	Features could be size and neighborhood; target is sale price.
2	It estimates class membership / class probabilities; the name does not determine the task type.
3	Clusters are discovered groups; class labels are known target categories supplied for supervised training.
4	Rewards give feedback about actions; a final win or loss can reward a long sequence of earlier moves.
5	Prompt and instructions, relevant conversation history, supplied or retrieved context, and generated output, subject to system-specific limits.
6	It predicts likely continuations and may produce fluent text without verifying each claim.
7	When answers need current or source-specific facts from documents; retrieval does not update weights.
8	Instruction, context, examples, persona, format, and tone; use only what helps.
9	A chain passes one step's result into a later prompt; a single prompt asks for the whole task at once.
10	Check accuracy against approved sources, audience fit, required format, and any safety or privacy constraints.

STUDY + INTERVIEW

How to learn from this guide

Follow the parts in order, but learn by recalling and practicing - not by reading passively.

A study routine that works

Step	What to do	Time guide
1. Recall	Before reading, write what you already know about the topic.	3-5 min
2. Learn	Read one short section and trace its diagram with your finger.	10-15 min
3. Rebuild	Close the PDF. Draw the diagram and explain the idea without looking.	5-10 min
4. Practice	Do a small exercise, notebook, or worked example.	15-25 min
5. Reflect	Write one takeaway, one confusion, and one interview answer.	3-5 min

A flexible six-week path

Week	Study in this PDF	Practice and milestone
1. Foundations	Parts 1-2: data, features, labels, task types, learning families.	Review Python basics and a small tabular dataset. Explain feature vs label.
2. Supervised ML	Classification and regression; fitting and held-out evaluation.	Train a baseline classifier and regressor. Keep a separate test set.
3. Reliable evaluation	Use the interview appendix: overfitting, leakage, metrics, error analysis.	Compare a baseline with one stronger model; explain which errors matter.
4. Networks and learning signals	Parts 2-4: RL, neural-network layers, training, deep learning.	Draw a neural network and the agent-action-reward loop from memory.
5. LLM foundations	Parts 4-6: tokens, next-token generation, prompts, RAG, fine-tuning.	Build prompts for one task; compare a grounded answer with an ungrounded one.
6. Build and interview	Parts 7-8 plus interview cases: limitations, design, and evaluation.	Present one small project in 3 minutes; answer a mock ML design question.

Before you start. You can learn the concepts without advanced mathematics. Python basics and comfort with tables help. For deeper ML, add probability, statistics, vectors and matrices, and the idea of derivatives gradually.

Pace. A practical starting rhythm is four or five sessions per week, around 30-60 minutes each. Extend a week when coding or evaluation is new to you.

Where to learn next

Start with visual explanations and exercises: [Google Machine Learning Crash Course](#). Good next step after Parts 1-4; includes regression, classification, data, neural networks, LLMs, production, and fairness.

Practice with small coding exercises: [Kaggle Learn: Intro to Machine Learning](#). Guided exercises cover model validation, overfitting, and building a first model.

Use a practical Python library: [scikit-learn User Guide](#). Look up supervised learning, clustering, preprocessing, model selection, and evaluation.

Continue to LLMs: [Hugging Face LLM Course](#). Continue after Python and basic deep-learning concepts; it progresses from Transformers to tokenizers and fine-tuning.

STUDY + INTERVIEW

Learn by building small projects

Keep the project small. The goal is to practice the full workflow and explain your choices.

A four-project ladder

Project	Question and method	What to measure or show
1. Classification	Predict a category such as spam / not spam or flower type. Start with a simple baseline, then compare a second model.	Confusion matrix; precision and recall when false positives and false negatives have different costs.
2. Regression	Estimate a numeric value such as a house price. Compare a mean-price baseline with linear regression.	MAE in the target's units; inspect large errors and compare on held-out data.
3. Clustering	Group unlabeled examples by similar features. Scale appropriate numeric inputs and try a simple clustering method.	A 2D plot can help inspect groups; explain that clusters are not automatically real categories.
4. Document assistant	Use RAG over a small set of permitted course notes. Retrieve relevant passages and answer with source references.	Check retrieval relevance, whether the answer is supported by passages, and what happens when no source answers.

Use the same experiment checklist each time

1. State the user problem and what a useful prediction or answer means.
2. Identify input features and, for supervised work, the target label.
3. Inspect examples for missing values, duplicates, imbalance, and suspicious shortcuts.
4. Split data to match the real use case; prevent leakage across train and test.
5. Build a simple baseline before trying a more complex model.
6. Choose a metric that matches the cost of errors; inspect examples the model gets wrong.
7. Change one thing at a time, record the result, and explain the limitation.
8. Write a short README: goal, data, method, metric, result, limits, and next step.

```
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

This is the basic fit-then-predict shape used by many supervised estimators. Preprocessing must also be learned from the training split only.

What to include in a small project report

Show	Explain
One result	Baseline and model score on held-out data, with the metric and units.
One error	A specific false positive, missed positive, or large numeric error and why it matters.
One limitation	A gap in data, assumptions, fairness, privacy, or expected real-world performance.

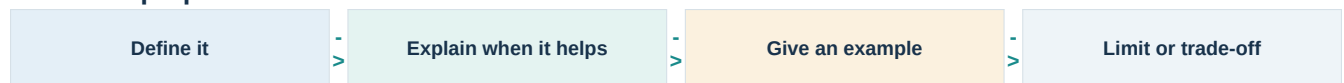
Portfolio rule. Show your reasoning and honest evaluation. A modest model with clean splits and clear error analysis is a stronger project story than an impressive score with unclear data handling.

STUDY + INTERVIEW

Answer ML interview questions clearly

A good answer is concise, concrete, and honest about assumptions and limitations.

For concept questions: DEAL



Aim for a 30-60 second first answer. Pause so the interviewer can ask a deeper follow-up.

Prompt	What a strong short answer includes
What is overfitting?	Definition, training vs validation behavior, one remedy, and why test performance matters.
Which metric would you use?	What counts as a costly error, the class balance, the metric, and the chosen threshold.
Why this model?	A simple baseline, the data/task properties, a validation comparison, and a limitation.

For ML system-design questions



Follow the decisions in order; do not jump straight to naming an algorithm.

Step	Questions to ask yourself
1. Goal	Who uses the prediction? What decision changes? What does success mean?
2. Data and target	What examples exist? Is the target available and trustworthy? Are there privacy or representational gaps?
3. Evaluation design	What data split reflects real use? Could time, users, or duplicate records leak across splits?
4. Baseline and model	What simple approach should be beaten? What model complexity is justified?
5. Metric and errors	Which mistakes are costly? How does performance vary across subgroups or conditions?
6. Delivery	How are predictions used, monitored for drift, corrected, and safely reviewed?

Pick a metric by asking what error costs more

Metric	Plain interpretation	Useful caution
Accuracy	Fraction of all predictions that are correct.	Can look high when the positive class is rare.
Precision	Of predicted positives, how many are truly positive?	Important when false alarms are costly.
Recall	Of actual positives, how many did the model find?	Important when missing a positive is costly.
F1	A combined precision-recall score at a chosen threshold.	Does not express every business cost; inspect both precision and recall.
ROC-AUC / PR-AUC	Measure ranking quality across thresholds; PR-AUC focuses on positive-class precision and recall.	For rare positives, PR-AUC can be more informative. Neither chooses the operating threshold for you.
MAE / RMSE	Average absolute error / square-rooted average squared error for regression.	RMSE penalizes large errors more; report units and compare to a baseline.

STUDY + INTERVIEW

Core machine-learning interview questions and answers

Practice saying the answer in your own words. These are starting answers, not scripts to memorize.

Question	Good first answer
What is generalization?	Performing well on new examples drawn from the intended use setting, not just on training data.
What is overfitting?	The model fits training-specific details or noise, so training performance is much better than validation performance. Try simpler models, regularization, more representative data, or early stopping.
What is underfitting?	The model is too limited or poorly trained to capture the useful pattern; training and validation performance are both weak.
What is data leakage?	Information unavailable at prediction time influences training or evaluation, making a score look unrealistically good. Split correctly and fit preprocessing only on training folds.
Why use a baseline?	It gives a simple reference score. A complex model is useful only if it improves the right metric enough to justify its cost and complexity.
Train vs validation vs test?	Train learns parameters; validation selects settings; test is a final held-out estimate. Repeatedly tuning to the test set turns it into validation data.
Classification vs regression?	Classification predicts a category; regression predicts a numeric value. Choose the task from the target, not the algorithm's name.
Why can accuracy fail?	With imbalanced classes, always predicting the majority class can produce high accuracy while missing most important positives. Compare precision, recall, and a baseline.
Precision vs recall?	Precision asks how many predicted positives are correct; recall asks how many actual positives were found. Their trade-off depends on false-positive and false-negative costs.
What is regularization?	A training penalty or constraint that discourages overly complex fits; it can reduce overfitting, but too much can cause underfitting.
Bias vs variance?	High bias comes from a model that is too restrictive; high variance means it changes too much with the training sample. Use validation behavior and the use case to guide the trade-off.
What is class imbalance?	One class is much rarer than another. Inspect class counts, use suitable metrics and stratified splits when appropriate, and choose a threshold based on error costs.
When does feature scaling matter?	Comparable feature scales often help distance-based and margin-based methods such as SVMs. Tree models are usually less sensitive. Learn scaling values from training data only.
How would you handle missing values?	Understand why values are missing, fit an imputer on training data, compare an appropriate baseline, and consider whether a missingness indicator is useful.
What is cross-validation?	Repeated train/validation splits estimate how a model performs across different subsets. Keep a separate final test set when a final estimate is needed.
What is a confusion matrix?	A count of true positives, false positives, true negatives, and false negatives; it shows which kinds of classification errors occur.
What is bias-variance trade-off?	A simpler model can miss real patterns; a very flexible model can fit noise. Compare train and validation behavior to choose useful complexity.
Why use a pipeline?	It keeps preprocessing and model steps together, helping ensure transformations are fit on training folds and repeated consistently at prediction time.

Leakage example. If a model predicts whether a customer will cancel next month, a feature containing next month's cancellation date leaks the answer. In time-based problems, train on the past and evaluate on later periods.

STUDY + INTERVIEW

Deep learning and GenAI interview questions and answers

Be ready to explain both the mechanism and when the method is useful.

Question	Good first answer
Logistic regression vs SVM?	Logistic regression models class probabilities with a decision boundary; an SVM seeks a maximum-margin boundary. Compare on validation data; scaling matters, and SVM scores need calibration to become probabilities.
What does clustering do?	It groups examples by a chosen similarity measure without target labels. A cluster is not automatically a meaningful real-world category.
What does dimensionality reduction do?	It represents many features with fewer dimensions to aid compression, visualization, or downstream modeling; some information may be lost.
Backpropagation vs gradient descent?	Backpropagation computes gradients of the loss through the network. An optimizer such as gradient descent uses gradients to update parameters.
What is self-supervised learning?	Training targets are derived from the input data, such as predicting masked or next tokens. It is a common pretraining approach for language models.
How is RL different?	An agent learns a policy from action and reward feedback, often with delayed rewards; it is not just supervised labels with another name.
What is a token?	A tokenizer unit - often a word piece, but it can be a word, punctuation, or another fragment. Token count affects context use.
What is an embedding?	A learned vector representation of an item such as a word or passage. Similar vectors can help find related content, but similarity is not proof of truth.
How does an LLM generate text?	A typical causal LLM repeatedly predicts a distribution for the next token given context, then decodes a token. Fluent output can still be false.
What is few-shot prompting?	Include a few input-output examples in a prompt so the model can follow a demonstrated pattern. Examples do not add verified facts.
Why can prompt wording matter?	Small changes in wording, order, or examples can shift the response. Give clear instructions and test representative prompt variations.
Open weights vs open source?	Open weights can be downloaded; that alone does not make training data, code, documentation, or license rights open. Check what is actually released.
RAG vs fine-tuning?	RAG retrieves documents at inference and supplies them as context; fine-tuning continues training and changes model weights. Use retrieval for changing source facts, tuning for repeated behavior or style.
Does RAG eliminate hallucination?	No. Retrieval can miss relevant passages, and the model can misread or go beyond them. Test retrieval and answer grounding separately.
What is a knowledge cutoff?	An approximate limit of information in the model's training. Search or RAG may supply newer facts for a request, but does not automatically update the model weights.
How do you request structured output?	Specify the required fields and format, then validate the result with a parser or schema. A prompt alone cannot guarantee valid JSON.
How do you evaluate an LLM app?	Test representative questions, difficult cases, source retrieval, factual grounding, citations, privacy controls, and failure behavior. Use human review where impact is high.
How would you address bias?	Define the relevant fairness concern for the use case, inspect data and subgroup outcomes, test proxies, and involve review. No single metric or sensitive-feature removal solves every bias.
What should never go into an unapproved prompt?	Confidential or sensitive data. Use approved tools, minimize data, and follow access and retention rules.

Careful with explanations. A model's written step-by-step rationale can help a reader inspect an answer, but it is not a correctness certificate or guaranteed trace of internal computation.

STUDY + INTERVIEW

Practice two interview cases

Use the answer framework, state assumptions, and connect model quality to the decision.

Case 1: design an email spam classifier

Interview step	A strong direction
Clarify	Does the goal minimize dangerous spam, reduce inbox clutter, or both? What is the cost of a legitimate email being blocked?
Data / labels	Use historical messages labeled by trusted review; check label quality, duplicates, language coverage, and privacy.
Baseline / model	Start with a simple rule or text classifier, then compare a model such as logistic regression. Avoid using features unavailable at delivery time.
Metric	Track precision and recall, plus false-positive examples. Choose the operating threshold from the cost of blocking wanted email.
Launch / monitor	Use a review or appeal route, monitor drift and subgroup behavior, and retrain only after checking fresh labels.

Case 2: build an internal policy-document assistant

Interview step	A strong direction
Clarify	Who may ask what? Which policy versions are authoritative? Must the assistant abstain when no source supports an answer?
Retrieve	Index approved documents, preserve permissions, retrieve relevant passages, and pass only authorized passages to the model.
Answer	Ask the model to answer from retrieved evidence and cite the passages; say when evidence is missing or conflicting.
Evaluate	Test retrieval coverage and answer groundedness separately. Include outdated, ambiguous, adversarial, and unanswerable questions.
Operate safely	Log minimally, protect private documents, review access, monitor changes, and provide a human escalation path.

Tell a truthful project story



Use this order for a 2-3 minute project answer. Say what you personally did and distinguish measured results from goals.

Practice questions: What baseline did you compare against? What data leakage did you rule out? Why that metric? Which errors mattered most? What would change if the data distribution shifted? What would you improve next?

Interview-ready checklist. Can I define the idea simply? Give a concrete example? Choose and justify a metric? Name one failure mode? Explain what I would test next? If not, revisit that section and practice aloud.

REFERENCE

Compact glossary and further study

Use these sources to explore the concepts in more depth.

Term	Meaning
Agent	The decision-maker in reinforcement learning.
Attention	A Transformer computation that weighs relationships among tokens.
Context window	The model's limited token context for a request and generated response.
Feature	An input clue or measurement used to make a prediction.
Fine-tuning	Further training that changes model parameters using selected examples.
Foundation model	A broadly pretrained model that can support many downstream uses.
Hallucination	A plausible-sounding but false or unsupported generated claim.
Label / target	The answer value a supervised model learns to predict.
LLM	A large language model trained to process and generate language.
Loss	A numerical measure of model prediction error for a training objective.
Policy	A strategy mapping an agent's situation to an action.
Prompt	Instructions and context supplied to guide an AI response.
RAG	Retrieval-Augmented Generation: retrieve source passages, then generate using them.
Token	A unit of text processed by a model; it may be smaller than a word.
Tokenizer	A program that converts text into model tokens / IDs.
Weight	A learned numerical parameter in a model.

Selected references

1. Scikit-learn User Guide - supervised and unsupervised learning
2. OpenAI Spinning Up - key concepts in reinforcement learning
3. Vaswani et al. (2017), Attention Is All You Need
4. Lewis et al. (2020), Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks
5. Wei et al. (2022), Chain-of-Thought Prompting Elicits Reasoning in LLMs
6. Lanham et al. (2023), Measuring Faithfulness in Chain-of-Thought Reasoning
7. Liu et al. (2022), Generated Knowledge Prompting for Commonsense Reasoning
8. Open Source Initiative - Open Source AI Definition
9. Hugging Face LLM Course - tokenization
10. Hugging Face Transformers - causal language modeling
11. Hugging Face Transformers - masked language modeling

Final reminder Use models as helpers for learning and production. Match the method to the task, provide relevant context, and verify important outputs.